EECE2310 Introduction to Digital Design and Computer Architecture
Summer 2 2025 Semester

Homework 2

EECE2310 Course Section 01

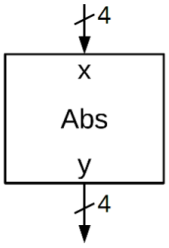
Name: Hang Hang

[redacted]

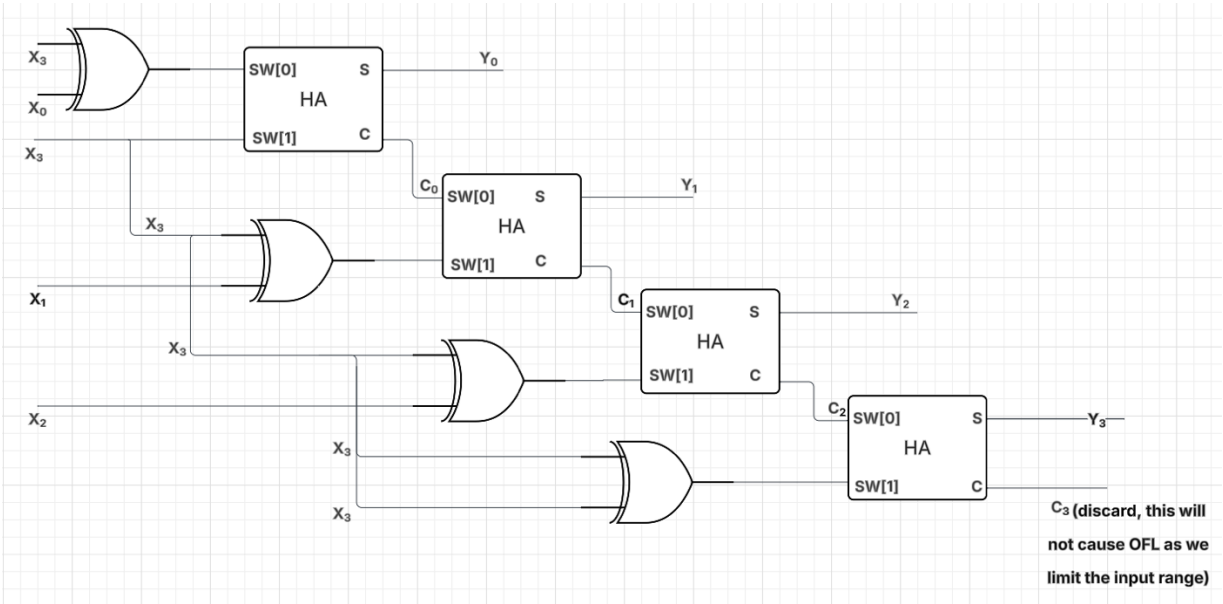
Date Submitted:
7/15/2025

Q1 (20 Points)

By using only four *Half Adders* and four *XOR* gates, implement the shown **Abs** combinational circuit that takes a signed 4-bit binary number **X** ($X_3X_2X_1X_0$ represented using 2's complement) as input and outputs **X**'s absolute value **Y** ($Y_3Y_2Y_1Y_0$). (Note: **Y=X** if **X** is positive and **Y= -X** if **X** is negative). Assume the $X_3X_2X_1X_0$ input is only limited to the range from -7 to +7.



Abs combinational circuit:



Q2 (15 Points)

Use only one block of a 3 × 8 *Decoder* and three *OR* gates to implement the following three Boolean functions (one circuit to implement all of them):

$$F_1 = x'yz' + xz$$
$$F_2 = xy'z' + x'y$$
$$F_3 = x'y'z' + xy$$

minterm	x	y	z	Terms
m0	0	0	0	x'y'z'
m1	0	0	1	x'y'z
m2	0	1	0	x'yz'
m3	0	1	1	x'yz
m4	1	0	0	xy'z'
m5	1	0	1	xy'z
m6	1	1	0	xyz'
m7	1	1	1	xyz

Expanding the given function:

F1: expand xz as the first term x'yz' already has all variables

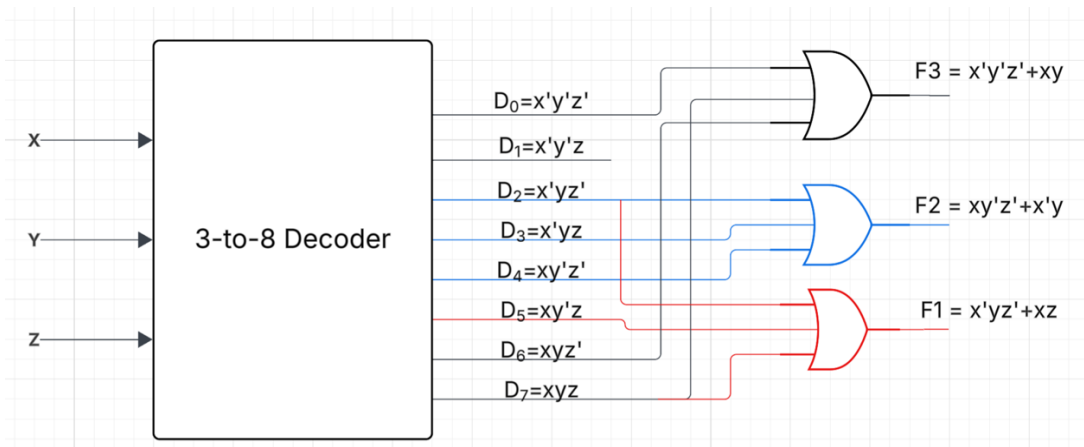
So F1=x'yz'+xz(y+y')=x'yz'+xyz+xy'z= **Σm(2, 5, 7)** → F1 = D2 + D5 + D7

F2: expanding x'y as the term xy'z' already has all variables

So F2= xy'z'+x'y=xy'z'+x'y(z+z')=xy'z'+x'yz+x'yz'= **Σm(2, 3, 4)** → F2 = D2 + D3 + D4

F3: expanding xy as the term x'y'z' already has all variables

So F3= x'y'z'+xy = x'y'z'+ xy(z+z')=x'y'z'+xyz+xyz'= **Σm(0, 6, 7)** → F3 = D0 + D6 + D7



Q3 (20 Points)

Implement the following Boolean function with only one 4 × 1 *Multiplexer*, one *OR* gate, and one *AND* gate.

$$F(A, B, C, D) = \sum m(1, 3, 5, 6, 7, 11, 12, 13, 14, 15)$$

A 4×1 multiplexer has:

- 4 input line (I_0, I_1, I_2, I_3)
- 2 selection lines (Use A= S_1 and B=S_0)
- Output = 1 of the 4 inputs based on the select lines

S_1 (A)	S_0 (B)	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Minterm	**A**	**B**	**C**	**D**	**F**
1	0	0	0	1	1

Minterm	A	B	C	D	F
3	0	0	1	1	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	1

Based off on the minterm table:

when A=0, B=0 (I_0)

- F=1 for m_1, m_3
- (C=0, D=1) OR (C=1, D=1)
- $I_0 = D$ (logic low/GND)

when A=0, B=1 (I_1)

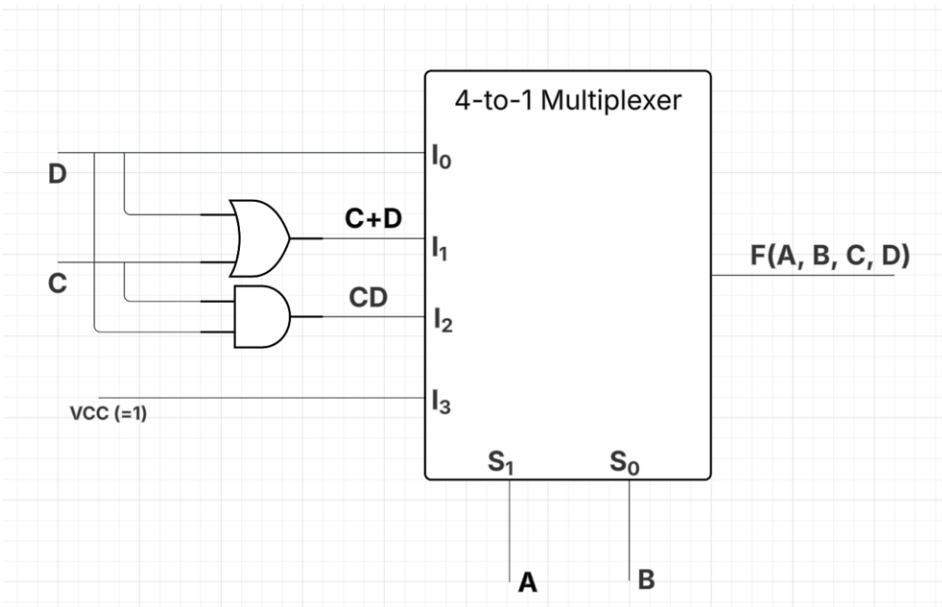
- F=1 for m_5, m_6, m_7
- (C=0, D=1) OR (C=1, D=0) OR (C=1, D=1)
- $I_1 = C+D$

when A=1, B=0 (I_2)

- F=1 for m_{11}
- C=1, D=1
- $I_2 = C \cdot D$

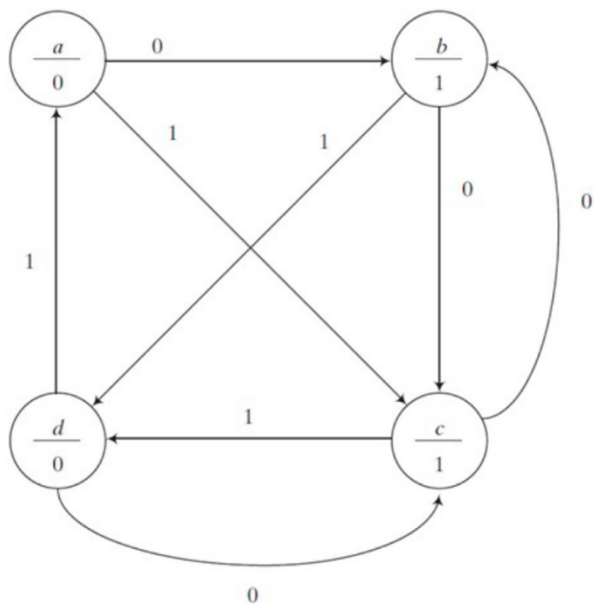
when A=1, B=1 (I_3)

- F=1 for m_{12}, m_{13}, m_{14}, m_{15}
- F=1 for all combinations of CD
- $I_3 = 1$ (logic high/VCC)



Q4 (25 Points)

By only using the minimum number of *D* Flip Flops and K-maps to find its combinational circuits, design a sequential circuit that implements the shown state diagram. The circuit has a one-bit input and a one-bit output. The binary number on the arrows represents the input value and the binary number inside the state circles represents the output value. You must assign the following 2-bit binary codes to the states: *a*=00, *b*=01, *c*=10, *d*=11.



We got four states: a, b, c, d
Since $2^n \geq$ number of states
n=2 --> we need **<u>2 D Flip Flops</u>**

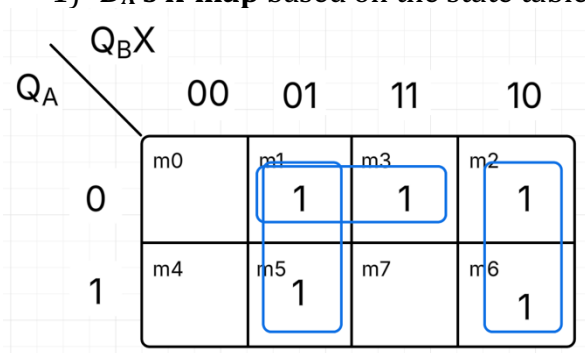
<u>State Table</u>

Current State		Input	Next State	
Q_A	Q_B	x	D_A	D_B
0	0	0	0	1
0	0	1	1	0

0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	0
1	1	1	0	0

We need D_A, D_B, output y for drafting the sequential circuit

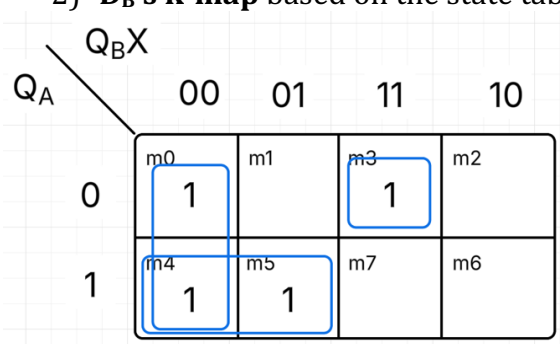
1) **D_A's k-map** based on the state table



Q_A Q_B x	D_A
000	0
001	1
010	1
011	1
100	0
101	1
110	1
111	0

$D_A = Q_B'x + Q_A'x + Q_Bx'$

2) **D_B's k-map** based on the state table

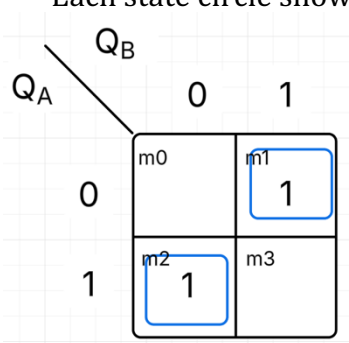


Q_A Q_B x	D_B
000	1
001	0
010	0
011	1
100	1
101	1
110	0
111	0

$D_B = Q_B'x' + Q_AQ_B' + Q_A'Q_Bx$

3) **y's k-map**

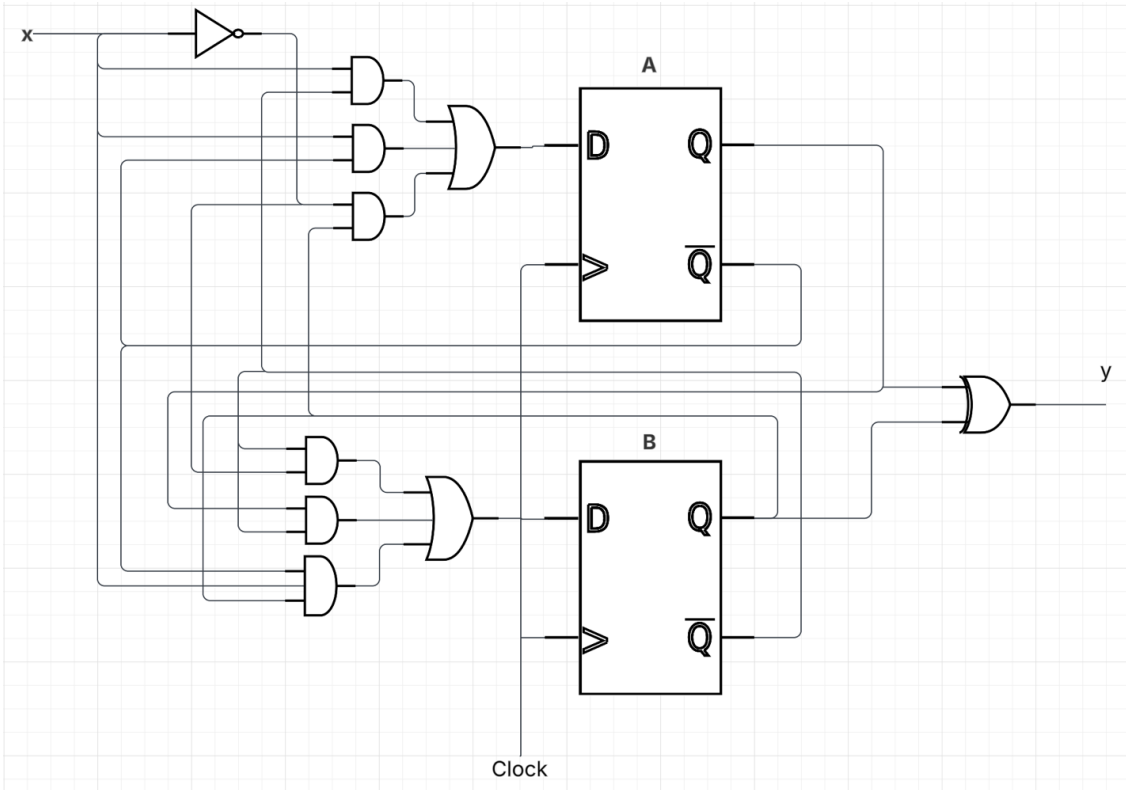
Each state circle shows current state/output value (a/0-> output=0, etc.)



Q_A Q_B	y
00	0
01	1
10	1
11	0

$y = Q_A Q_B' + Q_A' Q_B = Q_A \oplus Q_B$

Sequential Circuit Design:

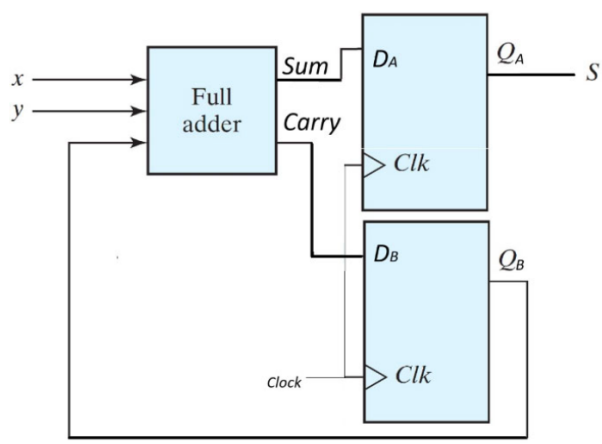


Q5 (20 Points)

The shown sequential circuit has two *D* flip-flops, two 1-bit inputs *x* and *y*, and one 1-bit output *S*. Its next state is determined by a full-adder circuit as shown.

a) Derive the state table and state diagram of the sequential circuit.

b) What is the maximum size, in bits, of the two integers this circuit can help adding? (i.e., If you want to use this circuit to add two integers, what is the restriction the circuit puts on the size of these integers?).



a)

Full Adder Truth Table (2^3 = 8 rows)

x	y	Carry-in (Q_B)	Sum (D_A)	Carry (D_B)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

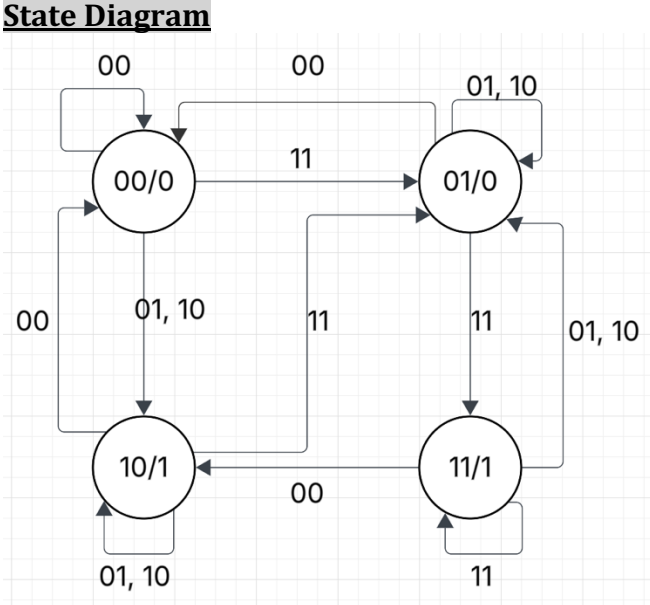
State Table: # of entries = 2^(m+n) = 2^(2+2) =16

Current State		**Inputs**		**Next State**	
Q_A	Q_B	x	y	D_A	D_B
0	0	0	0	0	0
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	0	1
0	1	1	0	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	1	0
1	0	1	0	1	0
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	1	1

Output Truth Table

Current State		Output
Q_A	Q_B	$S_{current}$
0	0	0
0	1	0
1	0	1
1	1	1

State Diagram



b) The maximum size restriction is 1 bit per integer operand. Total bit budget is 1 bit+1 bit=2 bits overall.

Explanation: While the circuit processes 3 bits per clock cycle (x + y + carry from QB), the actual operand inputs are constrained by:

- Input x is 1-bit wide (0 or 1)
- Input y is 1-bit wide (0 or 1)
- The carry from Q_B is internal state, not external input

The circuit limits the external input to 1-bit operands, though it can process arbitrarily large numbers through bit-by-bit addition over multiple cycles. For larger numbers, the circuit operates as a serial adder, breaking it down bit-by-bit, processing from LSB to MSB while propagating carry between each cycle.

Example - adding 5+3 (101_2+011_2):
Cycle 1: Add LSBs: 1+1+0 (initial carry)=1, carry=1
Cycle 2: next bits: 0+1+1(carry)=0, carry=1
Cycle 3: MSBs: 1+0+1(carry)=0, carry=1, this final carry goes into the next cycle

Example of valid single cycle operations:
- 0 + 0 = 0, 1 + 0 = 1, 1 + 1 = 0 (+ any carry from prev cycle)

Example of invalid operations:
- 2 + 0 (requires 2 bits (10_2), exceeding the 1-bit limit)
- Or any decimal number ≥ 2 requires multi-bit representation